

Prompt Reinforcing for Long-term Planning of Large Language Models

Hsien-Chin Lin^{†,*}, Nurul Lubis[†], Benjamin Matthias Ruppik[†], Carel van Niekerk[†],
Chia-Hao Shen[‡], Michael Heck[†], Renato Vukovic[†], Shutong Feng[†], Milica Gašić[†]

[†]Heinrich-Heine-Universität Düsseldorf, [‡]Independent researcher

Abstract

Large language models (LLMs) have achieved remarkable success in a wide range of natural language processing tasks and can be adapted through prompting. However, they remain suboptimal in multi-turn interactions, often relying on incorrect early assumptions and failing to track user goals over time, which makes such tasks particularly challenging. Prior works in dialogue systems have shown that long-term planning is essential for handling interactive tasks. In this work, we propose a prompt optimisation framework inspired by reinforcement learning, which enables such planning to take place by only modifying the task instruction prompt of the LLM-based agent. By generating turn-by-turn feedback and leveraging experience replay for prompt rewriting, our proposed method shows significant improvement in multi-turn tasks such as text-to-SQL and task-oriented dialogue. Moreover, it generalises across different LLM-based agents and can leverage diverse LLMs as meta-prompting agents. This warrants future research in reinforcement learning-inspired parameter-free optimisation methods.

1 Introduction

Large language models (LLMs) have shown an extraordinary ability to perform a wide range of tasks, from generating images in various styles to writing code in different programming languages for diverse purposes. LLMs are typically post-trained using reinforcement learning from human feedback (RLHF) (Ouyang et al., 2022), where they receive single-turn rewards for individual responses rather than rewards reflecting the quality of an entire multi-turn conversation. This limits their effectiveness in interactions where tasks are underspecified and clarified over time, often leading to early mistakes, incorrect assumptions, and cascading failures (Laban et al., 2025). On the other hand, prior work in dialogue systems demonstrates that long-term planning is vital for interactive tasks, making it essential for LLMs (Young, 2002; Young et al., 2013).

Directly optimising model parameters can improve long-horizon behaviour (e.g., LoRA, DPO, and dialogue-level RL) (Hu et al., 2022; Feng et al., 2025b,a), but it is often too expensive for real-time updates and infeasible for API-only models.

Gradient-free self-feedback methods avoid weight updates (Peng et al., 2023; Shinn et al., 2023; Yao et al., 2023; Elizabeth et al., 2025), but often require repeated feedback loops during inference. Existing prompt-optimisation methods mostly target input-output tasks and do not explicitly address long-horizon multi-turn planning (Yang et al., 2024a; Tang et al., 2025; Pryzant et al., 2023; Yuksekgonul et al., 2025).

To address these limitations, we propose Reinforced Prompt Optimisation (RPO), illustrated in Figure 1. RPO improves long-horizon behaviour by iteratively refining the task instruction from natural-

*linh@hhu.de

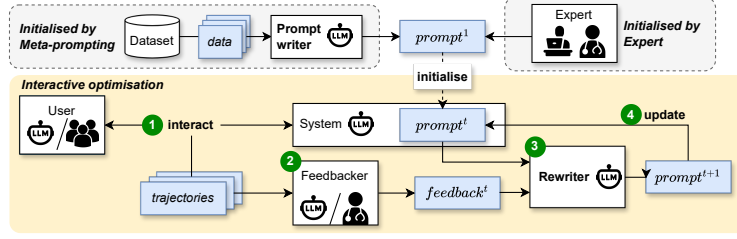


Figure 1: **Structure of Reinforced Prompt Optimisation (RPO)**. The initial $prompt^1$ can be written by LLMs or experts. In interactive optimisation, the system will first interact with the environment (simulated or real users). Then, the feedbacker (human or LLM) will provide textual feedback based on trajectories, and the rewriter generates a new prompt based on the original prompt and the textual feedback to update the system’s original prompt. One cycle of interactive optimisation is called an epoch, denoted by *superscripts*.

language feedback, starting from either expert-written or automatically generated prompts (Zhou et al., 2023; Pryzant et al., 2023; Ye et al., 2024).

In RPO, an LLM-based system interacts with an environment (real or simulated users). A feedbacker (human or LLM) provides turn-level textual feedback inspired by temporal difference (TD). As shown in the right part of Figure 2, for each turn t_i , the LLM-generated feedback includes: (1) predicted user emotion in the next turn elicited by the system response a_i , (2) a forecast of dialogue success or failure, and (3) suggestions based on the subdialogue $t_{1:i}$. These are then aggregated into dialogue-level feedback.

A separate LLM-based rewriter refines the prompt based on the feedback and the previous prompt. Experience replay is applied by leveraging feedback–prompt pairs from both the current and past iterations. The updated prompt is used in future interactions (Section 3). Inspired by these well-studied reinforcement learning concepts, the goal of RPO is to effectively strengthen the system agent’s long-term planning ability and overall task success.

Our contributions are as follows:

- We propose Reinforced Prompt Optimisation (RPO), a meta-prompting framework that improves multi-turn planning by iteratively updating task instructions from textual feedback.
- We study two design choices for feedback-driven prompt updates: turn-level TD-style feedback and replay-based rewriting.
- RPO works with external expert signals and generalises across both system backbones and meta-prompting LLMs, including API-only settings.

2 Related work

Gradient-based optimisation for LLMs Full fine-tuning of large models is expensive, so parameter-efficient methods (e.g., adapters and LoRA) are widely used (Hu et al., 2022, 2023; Lialin et al., 2023). Continuous prompting methods such as prefix-tuning and soft prompts can also approach fine-tuning performance (Lester et al., 2021; Qin and Eisner, 2021; Li and Liang, 2021; Liu et al., 2023, 2022). However, these methods are not applicable to API-only models and are hard to deploy for rapid online updates.

Self-feedback To improve the performance of text-based prompts, various prompting styles are proposed, e.g., Chain-of-Thought (Wei et al., 2022) or ReAct (Yao et al., 2023). These prompting methods encourage LLMs to reason before taking action or generating responses, which leads to better performance. However, optimising the prompt for better performance by manual trial and error is inefficient. Instead, self-feedback methods are introduced to refine the LLMs’ response, e.g., LLM-augmenter generates feedback by itself and leverages external knowledge to rewrite its response (Peng et al., 2023), and Reflexion summarises previous interactions with the environment as ‘reflections’ to improve the model’s response (Shinn et al., 2023; Madaan et al., 2023).

While this demonstrates the ability of LLMs for self-correction, these self-feedback methods rely on frequent API calls since their original prompt is not optimal. As a result, the computation cost and latency during inference are not negligible.

Prompt optimisation Meta-prompting methods are widely used to generate a prompt without human editing. The automatic prompt engineer (APE) method leverages an LLM, which is instructed to generate an initial prompt and selects the prompt with the best performance on the target task (Zhou et al., 2023). Automatic prompt optimisation (APO) further employs a self-feedback module to provide textual feedback, which gives suggestions on how to edit the old prompt (Pryzant et al., 2023). Ye et al. (2024) propose a meta-prompt LLM to edit the original prompt step-by-step. Kong et al. (2024) and Cheng et al. (2024) train a sequence-to-sequence model for prompt rewriting by reinforcement learning and preference data, respectively. Yang et al. (2024a) propose optimisation by prompting (OPRO), which leverages LLMs to rewrite the original prompt based on a corresponding performance score. To leverage experience, Zhang et al. (2023) model LLMs as semi-parametric RL agents with memory storing task data, actions, and Q -value estimates for few-shot in-context learning. Zhang et al. (2024) propose Agent-Pro, which constructs policy-level reflections according to the numerical feedback from the environment and improves its policy incrementally. Tang et al. (2025) introduce the Gradient-inspired LLM-based Prompt Optimizer (GPO), which updates the prompt iteratively based on numerical feedback and controls the edit distance through a cosine-based decay strategy. TextGrad generates textual feedback based on the user input and system output for prompt rewriting (Yuksekgonul et al., 2025). Although these methods demonstrate promising performance in generating or improving prompts, they focus on single-turn tasks. Our approach addresses multi-turn interactions, where prompts are updated with temporally grounded feedback to enhance long-term planning ability.

Learning ability of LLMs via prompting Although transformers are universal approximators (Yun et al., 2020) and in-context learning in LLMs can be viewed as implicit fine-tuning (Dai et al., 2023), the following remain open questions: Can we prompt LLMs for arbitrary tasks, and what are the limitations of in-context learning?

Petrov et al. (2024) highlight the limitations of context-based fine-tuning methods, e.g., in-context learning, prompting, and prefix tuning, for new task learning in transformers. Specifically, transformers struggle to acquire new tasks solely through prompting, as prompts cannot change the model’s attention patterns. Instead, they can only bias the output of the attention layers in a fixed direction and elicit skills learned through pre-training. Namely, only models with billions of parameters trained on vast, diverse datasets are capable of in-context learning, adapting to new tasks through examples or instructions without modifying their underlying weights. Therefore, we focus on fundamental models large enough to demonstrate their in-context learning ability, to investigate reinforcement prompt optimisation, which is fully composed of in-context learning with LLMs.

3 Reinforced Prompt Optimisation

Inspired by the gradient-based optimisation and reinforcement learning algorithms, where a model is initialised from pretraining and then further updated by on-policy learning based on interactions with the environment, we propose the Reinforced Prompt Optimisation (RPO) method (as shown in Figure 1 and the pseudo code can be found in Algorithm 1). The initial instruction can be generated by a prompt writer LLM_P such as the automatic prompt engineer (APE) (Zhou et al., 2023) (the upper left part of Figure 1) or written by human experts (the upper right part of Figure 1).

In each optimisation epoch, the **system** interacts with an environment (human or simulated users) and produces multi-turn *trajectories*. The **feedback** (LLM_F or human expert) generates textual feedback, and the **rewriter** (LLM_R) updates the prompt accordingly.

Unlike self-refine methods that edit model outputs, RPO edits the task instruction itself. Treating the instruction as a textual parameter reduces serving overhead by avoiding repeated feedback loops at inference time.

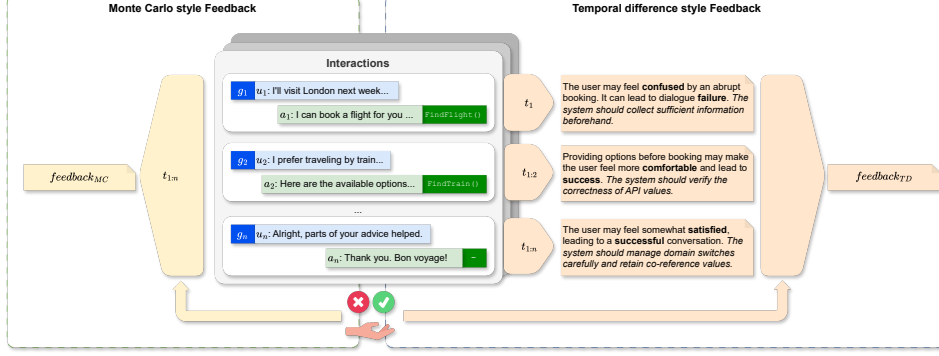


Figure 2: Workflow of feedback generation by an LLM. The Monte Carlo–style feedback (left) is generated after the entire interaction is completed, whereas the Temporal Difference–style feedback (right) consists of turn-level sub-feedback. Each turn includes natural-language utterances and, when available, the user’s goal (in blue) and API calls (in green). Each sub-feedback includes a justification, two predictions (next-turn user satisfaction and goal success), and an actionable suggestion.

3.1 Feedback generation

As shown in Figure 2, we consider two feedback schemes: Monte Carlo (MC)-style and Temporal Difference (TD)-style.

The **MC-style feedback** is produced only after the entire dialogue trajectory ($t_{1:n}$) has been completed (the prompt of the MC-styler feedbacker is shown in Figure 16):

$$feedback_{MC} = LLM_F(t_{1:n}) \quad (1)$$

This approach is common in single-turn tasks (Pryzant et al., 2023; Ye et al., 2024; Wang et al., 2024; Tang et al., 2025; Yuksekgonul et al., 2025) and yields edits from a global success/failure signal. It captures overall quality but collapses multi-turn interactions into a single outcome.

In contrast, our proposed **TD-style feedback** incorporates turn-level evaluations:

$$feedback_{TD,j} = LLM_F(t_1, feedback_{TD,1}, t_2, feedback_{TD,2}, \dots, t_j), \quad (2)$$

where $feedback_{TD,j}$ is the turn-level feedback at turn j . All turn-level feedback, $feedback_{TD,1:j}$, is then summarised into final dialogue-level feedback $feedback_{TD}$ (prompt details in Figure 17; examples in Figure 7, Figure 8, and Figure 9). Instead of waiting for dialogue completion, TD-style feedback provides incremental sentiment/success predictions and actionable suggestions.

TD-style feedback can be interpreted as combining short-term and long-term signals (Ghazarian et al., 2022):

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t) \quad (3)$$

where r_t is a short-term signal (e.g., user sentiment), $V(s_t)$ is the current value estimate, and $V(s_{t+1})$ is the estimated long-term value².

3.2 Applying Feedback to the Prompt

Unlike gradient-based optimisation, where gradients can be added or subtracted from model parameters, incorporating textual feedback into prompts is non-trivial. One cannot concatenate or remove arbitrary text from the original prompt without risking incoherence or loss of functionality. To address this, we introduce a basic *rewriter* LLM_R to apply textual feedback on the original prompt:

$$prompt^{i+1} = LLM_R(prompt^i, feedback^i), \quad (4)$$

where i denotes the epoch index. Its instruction is shown in Figure 14.

²For episodic tasks, γ can be treated as 1 (Sutton et al., 2018).

Inspired by experience **replay** in reinforcement learning (Andrychowicz et al., 2017), the rewriter can leverage not only the prompt and feedback from the current epoch, but also those from previous epochs (its instruction is shown in Figure 15):

$$\text{prompt}^{i+1} = \text{LLM}_R(\text{prompt}^i, \text{feedback}^i, \text{prompt}^{i-1}, \text{feedback}^{i-1}, \dots, \text{prompt}^1, \text{feedback}^1). \quad (5)$$

RPO reduces task-specific manual prompt engineering by automating prompt refinement from environment feedback (simulated or human). The feedback and rewriter prompts are task-agnostic and specified once; optimising those meta-prompts is left to future work.

4 Experiment Settings

In this study, we focus on iterative meta-prompting by leveraging textual feedback from the environment. We conduct experiments on three challenging human-machine interaction tasks that require multiple turns: Text-to-SQL, Task-oriented Dialogue, and Medical Question-answering (Section 4.1). An overview is shown in Figure 3. Our meta-prompting components are *task-agnostic* (Section 4.2). They are designed to optimise the prompt of interactive LLM-based systems (Section 4.3). All prompts are zero-shot in-context instructions³.

4.1 Tasks

	Objective ← → Subjective		
Task	Text-to-SQL	Task-oriented dialogue	Medical Question-Answering
User instruction	Let's find unpopular museum focus on museums that have not had any visitors at all return the name of the museum	You are looking for a place to stay, the hotel should be in the cheap price range and in the city centre . You also need to find a restaurant nearby .	I only have cough as a symptom. Please recommend Chinese medicine or a prescription .
Dataset	Lost_in_conversation based on Spider	MultiWOZ	Huatuo-26M & ShenNong-TCM
Evaluation	Database result exact match	Goal success	Safety, Professionalism, and Fluency by human expert

Figure 3: The summary of our experiment tasks.

Text-to-SQL Laban et al. (2025) proposed 6 tasks to study the performance drop of LLMs from fully-specified user queries to multi-turn interactions. The multi-turn, sharded instruction (e.g., Shard 1 conveys the high-level intent, and subsequent shards provide incremental clarifications) is partitioned based on the single-turn, fully-specified instruction from the original dataset. The largest decline occurs in the Text-to-SQL task, which we therefore select to study under different prompt optimisation methods, using instructions and databases from the Spider dataset (Yu et al., 2018).

In this task, the system agent receives a database schema at the start of the interaction and generates SQL queries from user queries in natural language. We evaluate closed-source LLMs (GPT-4o mini, Gemini-2.0-flash) and open-source LLMs (Llama-3.1-8B, Llama-3.1-70B, Llama-4-scout). The agent is optimised in the multi-sharded environment and evaluated by *functional accuracy*.

Task-oriented Dialogue To evaluate on a more realistic scenario, we conduct experiments on MultiWOZ 2.1 (Budzianowski et al., 2018; Eric et al., 2020), containing 10k human-to-human conversations on information-seeking, recommendations, and reservations across multiple domains. In this work, we focus on the attraction, hotel, restaurant, and train domains, under the ConvLab-3 framework (Zhu et al., 2023). Each user goal of the simulated user is a plain-text description, e.g., “You are looking for a place to stay, the **hotel** should be in the **cheap price range** and **in the city centre**. You also need to find a **restaurant nearby**.”

³Following Brown et al. (2020), this is in-context learning because task descriptions are provided as context, and zero-shot because no demonstrations are included.

The system agent is FnCTOD (Li et al., 2024), built with GPT-4o mini. FnCTOD has two prompts (state tracking via function calls and response generation), both optimised in our setting. Performance is measured by *success rate* using the rule-based evaluator in ConvLab-3.

Medical Question-Answering To evaluate our system in a more human-centred setting and how well prompting can improve the model’s performance in a domain that is not common in the pre-training data, we use two medical question-answering datasets: Huatuo-26M (Wang et al., 2025) and ShenNong-TCM (Wei Zhu and Wang, 2023), which are collected from the internet, e.g., encyclopedias, books, literature, and web corpus, or generated by an LLM based on a traditional Chinese medicine entity graph in Huatuo-26M and ShenNong-TCM, respectively. Simulated users act based on descriptions in plain text, related to general medicine or traditional Chinese medicine, e.g., “我食欲不振，如何改善？(I am experiencing a loss of appetite. How to get it back?)”.

The system agent is built with GPT-4o mini and relies solely on pre-training knowledge (no external knowledge base). At each epoch, an expert with degrees in general medicine and traditional Chinese medicine provides feedback on 10 interactions. For evaluation, three experts compare system pairs on 90 interactions per domain (general medicine and traditional Chinese medicine), based on safety, professionalism, and fluency (Yang et al., 2024b).

4.2 Meta-prompting components

In interactive optimisation, the feedbacker LLM_F and rewriter LLM_R use either closed-source LLMs (GPT-4o mini, Gemini-2.0-flash) or open-source LLMs (Llama-3.1-8B, Llama-3.1-70B, Llama-4-scout). More detail is shown in Table 3. Across different tasks, the prompts of LLM_F and LLM_R remain fixed, highlighting the task-independent role of these components.

4.3 Optimisation and Evaluation

We first collect interactions using the initial prompt and user instructions sampled from the training set. Due to context limits and to facilitate human feedback, the feedbacker processes 10 interactions per epoch. The rewriter then generates 2 new prompts based on the previous prompt and the feedback. New interactions are collected with each candidate prompt, and the one with the highest score on the validation set (based on automatic metrics or human experts, depending on the task) is chosen for the next iteration. We use 5 epochs for Text-to-SQL, 8 for Task-oriented Dialogue, and 3 for Medical Question-Answering.

Baselines In our experiments, we compare various prompt optimisation methods. *Black-Box Prompt Optimization (BPO)* fine-tunes Llama-2-7B-Chat (Touvron et al., 2023) for prompt rewriting based on preference learning (Cheng et al., 2024). *Automatic Prompt Optimisation (APO)* uses the user input, system output, and label to generate feedback (Pryzant et al., 2023). For multi-turn interactions, golden labels are infeasible since multiple solution paths exist; thus, we use a binary success/failure label. *Gradient-inspired Prompt Optimizer (GPO)* iteratively updates prompts via numerical feedback, e.g., functional accuracy for Text-to-SQL, task success for dialogue (Tang et al., 2025). *MC-style (TextGrad)* (Yuksekgonul et al., 2025) processes entire conversations to generate textual feedback (see Section 3.1).

5 Results and Discussion

5.1 Robustness and Generalisability

System agents as different LLMs Table 1 shows the results of optimising system agents built on five LLM backbones for the text-to-SQL task. Prompt optimisation methods aim to improve system agents in the multi-sharded setting, i.e., the user only reveals part of the information in one turn. For comparison, Oracle_{Full}, a single-turn setting where the user query is fully specified at once, is taken as an upper bound. The performance gap between Baseline_{Sharded} and Oracle_{Full} (average 0.333 vs. 0.743) highlights the difficulty LLMs face in handling multi-turn interactive tasks.

The prompts optimised by BPO do not improve performance; in some cases, they even degrade it, especially when applied to systems built on different model families, e.g. GPT-4o mini or Gemini-2.0-flash. It is not surprising that BPO is inferior since it is optimised for revising prompts for single-turn

tasks and does not generalise well to multi-turn scenarios. Its generalisability is further limited by its dependence on the Llama family as the backend, resulting in suggestions that may not transfer to GPT- or Gemini-based system agents. On the other hand, RPO_{TD} outperforms prior approaches when the system agent is built with Gemini-2.0-flash, Llama-4-scout, and Llama-3.1-70B. In contrast, $\text{RPO}_{\text{TD+replay}}$ achieves the best overall performance, with an average score of 0.477 (+54.2% over $\text{Baseline}_{\text{Sharded}}$). Llama-3.1-8B benefits the most, since its performance optimised by $\text{RPO}_{\text{TD+replay}}$ (0.467) nearly matches the oracle fully-specified setting (0.505). The gains across both closed-source and open-source backbones show strong robustness.

However, despite substantial gains over the sharded baseline, a gap to the baseline with the fully-specified user query (average 0.477 vs. 0.743) underscores that prompt optimisation can mitigate, but not fully eliminate, the degradation caused by multi-turn interactions.

Table 1: Functional accuracy with a 95% confidence interval of Text-to-SQL system agents built on five LLMs optimised with various methods. $\text{Oracle}_{\text{Full}}$: An oracle baseline in a single-turn setting with fully-specified user queries. The final two columns show the average score (Mean) and the relative improvement ($\Delta\%$) over the $\text{Baseline}_{\text{Sharded}}$. **Bold** scores are significantly better than the others ($p < 0.05$).

Method	LLM of the <i>system</i> agent					Mean	$\Delta\%$
	GPT	Gemini	Llama-4	Llama-8B	Llama-70B		
$\text{Baseline}_{\text{Sharded}}$	0.402 $\pm$ 0.002	0.514 $\pm$ 0.027	0.206 $\pm$ 0.013	0.224 $\pm$ 0.016	0.318 $\pm$ 0.014	0.333	-
BPO	0.318 $\pm$ 0.061	0.397 $\pm$ 0.069	0.220 $\pm$ 0.026	0.185 $\pm$ 0.028	0.241 $\pm$ 0.061	0.272	-15.7
APO	0.374 $\pm$ 0.009	0.523 $\pm$ 0.008	0.318 $\pm$ 0.012	0.290 $\pm$ 0.013	0.336 $\pm$ 0.004	0.368	16.9
GPO	0.458 $\pm$ 0.008	0.523 $\pm$ 0.016	0.299 $\pm$ 0.017	0.290 $\pm$ 0.015	0.308 $\pm$ 0.011	0.376	17.5
MC-style	0.459 $\pm$ 0.018	0.551 $\pm$ 0.015	0.250 $\pm$ 0.013	0.346 $\pm$ 0.021	0.332 $\pm$ 0.007	0.388	20.4
RPO_{TD} (ours)	0.439 $\pm$ 0.011	0.561 $\pm$ 0.013	0.336 $\pm$ 0.014	0.318 $\pm$ 0.009	0.383 $\pm$ 0.011	0.408	28.9
$\text{RPO}_{\text{TD+replay}}$ (ours)	0.528 $\pm$ 0.011	0.607 $\pm$ 0.018	0.383 $\pm$ 0.013	0.467 $\pm$ 0.022	0.402 $\pm$ 0.012	0.477	54.2
$\text{Oracle}_{\text{Full}}$	0.893 $\pm$ 0.007	0.841 $\pm$ 0.007	0.729 $\pm$ 0.017	0.505 $\pm$ 0.017	0.748 $\pm$ 0.014	0.743	140.2

Prompt optimisation with different LLMs Table 2 reports the success rates of FnCTOD (Li et al., 2024) when optimised by different prompt optimisation methods across five LLM backbones. The baseline system achieves a success rate of 0.420, while all optimisation methods substantially improve performance. Among prior approaches, MC-style feedback yields the strongest results with a mean success rate of 0.565 (+34.4% over baseline), slightly outperforming APO and GPO. Our proposed methods consistently surpass these baselines. In particular, RPO_{TD} achieves a mean score of 0.575 (+37.0%), demonstrating the advantage of trajectory-driven optimisation. When combined with the rewriter with experience replay, $\text{RPO}_{\text{TD+replay}}$ delivers the best performance across all LLMs, reaching an average success rate of 0.619, corresponding to a relative improvement of 47.3%. The gains are consistent across all five LLMs, confirming that RPO is robust and generalisable, independent of the underlying model of the meta-prompting agents.

Table 2: The success rate with a 95% confidence interval of the task-oriented dialogue system, FnCTOD (Li et al., 2024), improved by various prompt optimisation methods leveraging 5 different LLMs. The initial success rate of FnCTOD is 0.420. **Bold** scores are significantly better than the others ($p < 0.05$).

Method	LLM of the <i>meta-prompting</i> agent					Mean	$\Delta\%$
	GPT	Gemini	Llama-4	Llama-8B	Llama-70B		
APO	0.540 $\pm$ 0.030	0.560 $\pm$ 0.018	0.540 $\pm$ 0.018	0.560 $\pm$ 0.018	0.560 $\pm$ 0.018	0.552	31.4
GPO	0.579 $\pm$ 0.021	0.541 $\pm$ 0.008	0.571 $\pm$ 0.046	0.554 $\pm$ 0.017	0.526 $\pm$ 0.053	0.554	32.0
MC-style	0.567 $\pm$ 0.042	0.549 $\pm$ 0.010	0.575 $\pm$ 0.038	0.560 $\pm$ 0.029	0.572 $\pm$ 0.044	0.565	34.4
RPO_{TD} (ours)	0.578 $\pm$ 0.037	0.562 $\pm$ 0.036	0.586 $\pm$ 0.021	0.594 $\pm$ 0.013	0.556 $\pm$ 0.042	0.575	37.0
$\text{RPO}_{\text{TD+replay}}$ (ours)	0.625 $\pm$ 0.038	0.622 $\pm$ 0.018	0.618 $\pm$ 0.018	0.622 $\pm$ 0.007	0.606 $\pm$ 0.020	0.619	47.3

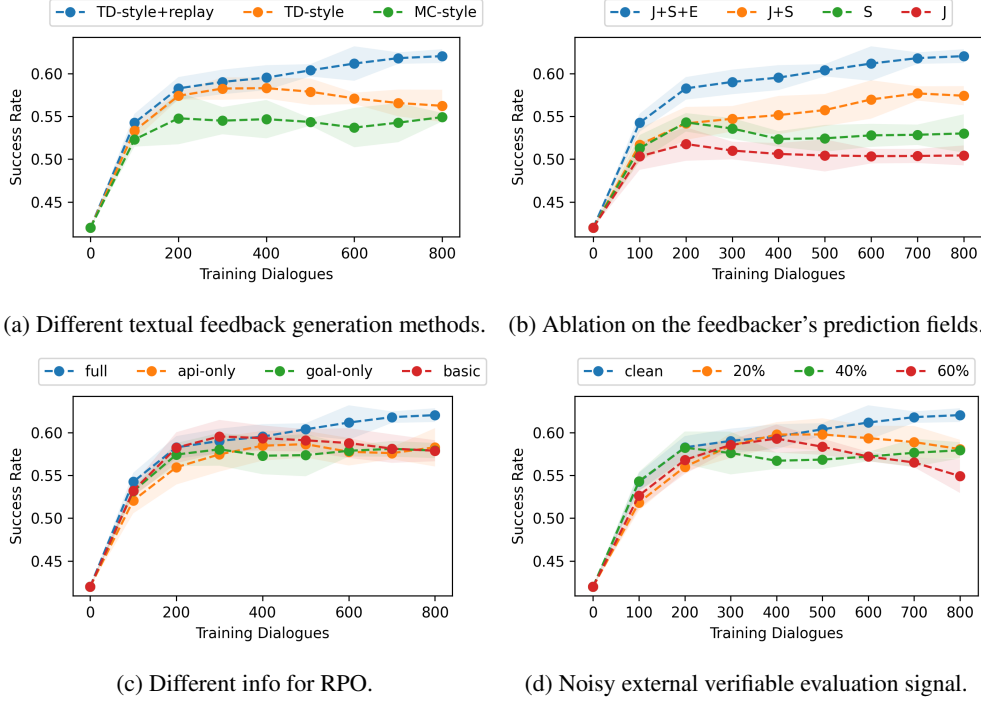


Figure 4: Each setting uses 4 training seeds and is evaluated on 100 dialogues; the line represents the mean success rate, and the shaded area indicates the standard error.

5.2 Ablation of feedback design in RPO

To examine whether the gains come from richer feedback design rather than generic iterative prompt refinement, we conduct four ablations: feedback style (MC vs. TD; Figure 4a), the prediction fields generated by the feedbacker (Figure 4b), input signals provided to the feedbacker (Figure 4c), and robustness to noisy external feedback (Figure 4d).

Feedback style and training curves The training curves of FnCTOD optimised by MC-style, TD-style, and TD-style+replay with Gemini-2.0-flash are shown in Figure 4a (results for other LLMs are shown in Figure 6). MC-style exhibits higher variance during early training, whereas TD-style is more stable and converges faster. With further training, their final performances become comparable. Incorporating experience replay into the rewriter further stabilises training and achieves the best overall performance. Qualitative comparisons of MC- and TD-style feedback, along with the resulting optimised prompts, are provided in Appendix E.

Prediction targets and performance gains We further ablate which *prediction fields* drive the gains, e.g. whether the short-term reward (next-turn user emotion) and long-term value (future task success) help. As shown in Figure 4b, no single target accounts for the observed gains: using only J or only S underperforms their combination ($J+S$), while the full setting ($J+S+E$) achieves the best performance. These results indicate that richer and more structured prediction targets are critical for effective feedback, beyond generic iterative prompt rewriting.

Effect of input signals on feedback quality We examine the impact of input signals provided to the feedbacker. The *basic* setting uses only the dialogue text. The *goal-only* setting additionally provides the user goal, while the *api-only* setting includes the system API call. The *full* setting (TD-style with replay) incorporates both signals. As shown in Figure 4c, both the user goal and the API call are essential for optimal performance. Although these signals can often be inferred implicitly, providing them explicitly leads to substantial improvements. This suggests that direct access to task-relevant state information enables the feedbacker to generate more specific and actionable feedback. Example prompts before and after optimisation with $RPO_{TD+replay}$ are shown in Figure 11 and Figure 13.

Robustness to noisy feedback To more substantiate the impact of the *quality of textual feedback*, we flipped 20%, 40%, and 60% of the external evaluation signal to assess robustness to noisy external evaluation signals. As shown in Figure 4d, the performance is robust when the noise level is up to 40%. The optimisation ability of RPO declines when the noise reaches 60%, which reverses the reward signal on average and naturally harms any optimisation. On the other hand, if the noise level is less than 50%, its performance is close to the clean setting. In other words, the turn-level feedback, which is not conditioned on the external feedback, provides a learning signal that, to some extent, mitigates the noise in the external evaluation signal.

5.3 Prompting limitations on underrepresented topics in LLMs

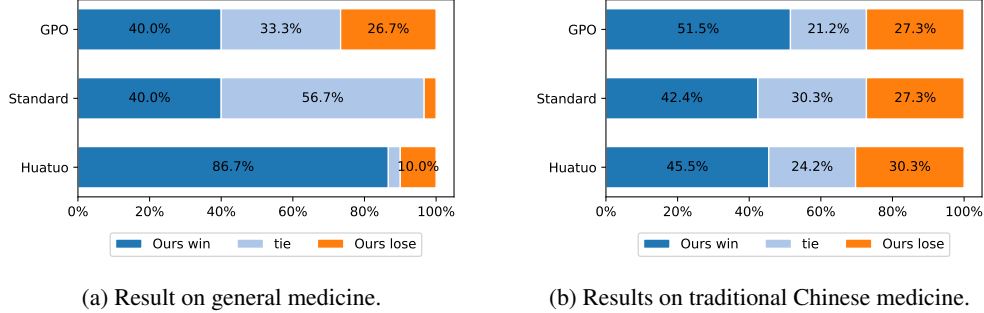


Figure 5: Overall preference between our method and a standard system (Standard), GPO, and HuatuoGPT-II (Huatuo) on the medical question-answering task. The overall recommendation by human experts is based on safety, professionalism, and fluency.

We compare our method against three systems: a standard system, built with GPT-4o mini with the initial prompt, a standard system updated via GPO, and HuatuoGPT-II (Chen et al., 2024), a large language model which is fully fine-tuned on medical data and demonstrates the state-of-the-art performance on Chinese medicine benchmarks. In other words, except HuatuoGPT-II, a fully fine-tuned 7B model, all systems are built with GPT-4o mini by prompting.

In general medicine, our method consistently outperforms the fully fine-tuned HuatuoGPT-II with an 86.7% win rate and is preferred over other prompting-based baselines (see Figure 5a). On the other hand, traditional Chinese medicine is more challenging. For example, our system’s preference rate drops by 41% compared to Huatuo when transitioning from general medicine to traditional Chinese medicine (see Figure 5b). However, despite this drop in preference, our proposed method is still favoured in general.

This observation is aligned with the findings by Petrov et al. (2024). Our method performs better in general medicine because the skills present in the pre-training data of LLMs can be elicited by prompting. However, tasks that are unseen or underrepresented in pre-training data are hard to learn through prompting. How to properly leverage external knowledge to improve the performance on unseen or under-represented tasks is an important future work.

6 Conclusions

We proposed a robust framework for interactive prompt optimisation that can effectively optimise system agents built on diverse LLM backbones and system architectures, ranging from standard input-output agents in text-to-SQL and medical QA to multi-stage agents in task-oriented dialogue that access external knowledge sources. In addition, it is flexible to the choice of LLM used for generating feedback and rewriting, as it works effectively with both closed-source LLMs (GPT-4o mini and Gemini-2.0-flash) and open-source LLMs (Llama variants). Turn-level feedback enriched with user status and API details, together with experience replay in rewriting, proved highly effective for stabilising and enhancing optimisation in multi-turn tasks.

By using the optimised prompt, the system can minimise the need for extensive self-feedback loops, reducing computational overhead and API call frequency during inference. Although the performance

optimised by our method still falls short of fully specified settings and unseen tasks remain difficult to optimise purely by prompting, our reinforcement learning-inspired method offers a stable, practical, and efficient approach for automatic prompt optimisation to reduce the challenges of unspecified multi-turn interactions, which could be valuable for future LLM research.

References

- Andrychowicz, M., Wolski, F., Ray, A., Schneider, J., Fong, R., Welinder, P., McGrew, B., Tobin, J., Pieter Abbeel, O., and Zaremba, W. (2017). Hindsight experience replay. *Advances in neural information processing systems*, 30.
- Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., and Amodei, D. (2020). Language Models are Few-Shot Learners.
- Budzianowski, P., Wen, T.-H., Tseng, B.-H., Casanueva, I., Ultes, S., Ramadan, O., and Gašić, M. (2018). MultiWOZ - a large-scale multi-domain Wizard-of-Oz dataset for task-oriented dialogue modelling. In Riloff, E., Chiang, D., Hockenmaier, J., and Tsujii, J., editors, *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 5016–5026, Brussels, Belgium. Association for Computational Linguistics.
- Chen, J., Wang, X., Ji, K., Gao, A., Jiang, F., Chen, S., Zhang, H., Dingjie, S., Xie, W., Kong, C., Li, J., Wan, X., Li, H., and Wang, B. (2024). HuatuoGPT-II, one-stage training for medical adaption of LLMs. In *First Conference on Language Modeling*.
- Cheng, J., Liu, X., Zheng, K., Ke, P., Wang, H., Dong, Y., Tang, J., and Huang, M. (2024). Black-box prompt optimization: Aligning large language models without model training. In Ku, L.-W., Martins, A., and Srikumar, V., editors, *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3201–3219, Bangkok, Thailand. Association for Computational Linguistics.
- Dai, D., Sun, Y., Dong, L., Hao, Y., Ma, S., Sui, Z., and Wei, F. (2023). Why can GPT learn in-context? language models secretly perform gradient descent as meta-optimizers. In Rogers, A., Boyd-Graber, J., and Okazaki, N., editors, *Findings of the Association for Computational Linguistics: ACL 2023*, pages 4005–4019, Toronto, Canada. Association for Computational Linguistics.
- Elizabeth, M., Veyret, M., Couceiro, M., Dušek, O., and Barahona, L. M. R. (2025). Exploring ReAct Prompting for Task-Oriented Dialogue: Insights and Shortcomings. In *Proceedings of the 15th International Workshop on Spoken Dialogue Systems Technology*, pages 143–153.
- Eric, M., Goel, R., Paul, S., Sethi, A., Agarwal, S., Gao, S., Kumar, A., Goyal, A., Ku, P., and Hakkani-Tur, D. (2020). MultiWOZ 2.1: A consolidated multi-domain dialogue dataset with state corrections and state tracking baselines. In Calzolari, N., Béchet, F., Blache, P., Choukri, K., Cieri, C., Declerck, T., Goggi, S., Isahara, H., Maegaard, B., Mariani, J., Mazo, H., Moreno, A., Odijk, J., and Piperidis, S., editors, *Proceedings of the Twelfth Language Resources and Evaluation Conference*, pages 422–428, Marseille, France. European Language Resources Association.
- Feng, S., Lin, H., Lubis, N., van Niekerk, C., Heck, M., Ruppik, B., Vukovic, R., and Gašić, M. (2025a). Emotionally Intelligent Task-oriented Dialogue Systems: Architecture, Representation, and Optimisation.
- Feng, Z., Wang, X., Wu, B., Zhong, W., Xu, Z., Cao, H., Zhao, T., Li, Y., and Wang, B. (2025b). Empowering LLMs in Task-Oriented Dialogues: A Domain-Independent Multi-Agent Framework and Fine-Tuning Strategy.
- Ghazarian, S., Hedayatnia, B., Papangelis, A., Liu, Y., and Hakkani-Tur, D. (2022). What is wrong with you?: Leveraging User Sentiment for Automatic Dialog Evaluation. In Muresan, S., Nakov, P., and Villavicencio, A., editors, *Findings of the Association for Computational Linguistics: ACL 2022*, pages 4194–4204, Dublin, Ireland. Association for Computational Linguistics.

- He, H. and Thinking Machines Lab (2025). Defeating Nondeterminism in LLM Inference. *Thinking Machines Lab: Connectionism*. <https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/>.
- Hu, E. J., yelong shen, Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. (2022). LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*.
- Hu, Z., Wang, L., Lan, Y., Xu, W., Lim, E.-P., Bing, L., Xu, X., Poria, S., and Lee, R. (2023). LLM-adapters: An adapter family for parameter-efficient fine-tuning of large language models. In Bouamor, H., Pino, J., and Bali, K., editors, *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 5254–5276, Singapore. Association for Computational Linguistics.
- Kong, W., Hombaiah, S., Zhang, M., Mei, Q., and Bendersky, M. (2024). PRewrite: Prompt rewriting with reinforcement learning. In Ku, L.-W., Martins, A., and Srikumar, V., editors, *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 594–601, Bangkok, Thailand. Association for Computational Linguistics.
- Laban, P., Hayashi, H., Zhou, Y., and Neville, J. (2025). LLMs Get Lost In Multi-Turn Conversation.
- Lester, B., Al-Rfou, R., and Constant, N. (2021). The power of scale for parameter-efficient prompt tuning. In Moens, M.-F., Huang, X., Specia, L., and Yih, S. W.-t., editors, *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 3045–3059, Online and Punta Cana, Dominican Republic. Association for Computational Linguistics.
- Li, X. L. and Liang, P. (2021). Prefix-tuning: Optimizing continuous prompts for generation. In Zong, C., Xia, F., Li, W., and Navigli, R., editors, *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 4582–4597, Online. Association for Computational Linguistics.
- Li, Z., Chen, Z., Ross, M., Huber, P., Moon, S., Lin, Z., Dong, X., Sagar, A., Yan, X., and Crook, P. (2024). Large Language Models as Zero-shot Dialogue State Tracker through Function Calling. In Ku, L.-W., Martins, A., and Srikumar, V., editors, *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 8688–8704, Bangkok, Thailand. Association for Computational Linguistics.
- Lialin, V., Deshpande, V., and Rumshisky, A. (2023). Scaling down to scale up: A guide to parameter-efficient fine-tuning. *arXiv preprint arXiv:2303.15647*.
- Liu, X., Ji, K., Fu, Y., Tam, W., Du, Z., Yang, Z., and Tang, J. (2022). P-tuning: Prompt tuning can be comparable to fine-tuning across scales and tasks. In Muresan, S., Nakov, P., and Villavicencio, A., editors, *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 61–68, Dublin, Ireland. Association for Computational Linguistics.
- Liu, X., Zheng, Y., Du, Z., Ding, M., Qian, Y., Yang, Z., and Tang, J. (2023). Gpt understands, too. *AI Open*.
- Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegrefe, S., Alon, U., Dziri, N., Prabhunoye, S., Yang, Y., Gupta, S., Majumder, B. P., Hermann, K., Welleck, S., Yazdanbakhsh, A., and Clark, P. (2023). Self-refine: Iterative refinement with self-feedback. In Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., and Levine, S., editors, *Advances in Neural Information Processing Systems*, volume 36, pages 46534–46594. Curran Associates, Inc.
- Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., et al. (2022). Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730–27744.
- Peng, B., Galley, M., He, P., Cheng, H., Xie, Y., Hu, Y., Huang, Q., Liden, L., Yu, Z., Chen, W., et al. (2023). Check your facts and try again: Improving large language models with external knowledge and automated feedback. *arXiv preprint arXiv:2302.12813*.

- Petrov, A., Torr, P., and Bibi, A. (2024). When Do Prompting and Prefix-Tuning Work? A Theory of Capabilities and Limitations. In *The Twelfth International Conference on Learning Representations*.
- Pryzant, R., Iter, D., Li, J., Lee, Y., Zhu, C., and Zeng, M. (2023). Automatic prompt optimization with “gradient descent” and beam search. In Bouamor, H., Pino, J., and Bali, K., editors, *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 7957–7968, Singapore. Association for Computational Linguistics.
- Qin, G. and Eisner, J. (2021). Learning how to ask: Querying LMs with mixtures of soft prompts. In Toutanova, K., Rumshisky, A., Zettlemoyer, L., Hakkani-Tur, D., Beltagy, I., Bethard, S., Cotterell, R., Chakraborty, T., and Zhou, Y., editors, *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 5203–5212, Online. Association for Computational Linguistics.
- Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., and Yao, S. (2023). Reflexion: language agents with verbal reinforcement learning. In Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., and Levine, S., editors, *Advances in Neural Information Processing Systems*, volume 36, pages 8634–8652. Curran Associates, Inc.
- Sutton, R. S., Barto, A. G., et al. (2018). Reinforcement learning: An introduction 2nd ed. *MIT press Cambridge*, 1(2):25.
- Tang, X., Wang, X., Zhao, W. X., Lu, S., Li, Y., and Wen, J.-R. (2025). Unleashing the potential of large language models as prompt optimizers: analogical analysis with gradient-based model optimizers. In *Proceedings of the Thirty-Ninth AAAI Conference on Artificial Intelligence and Thirty-Seventh Conference on Innovative Applications of Artificial Intelligence and Fifteenth Symposium on Educational Advances in Artificial Intelligence, AAAI’25/IAAI’25/EAAI’25*. AAAI Press.
- Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., et al. (2023). Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*.
- Wang, X., Li, C., Wang, Z., Bai, F., Luo, H., Zhang, J., Jojic, N., Xing, E., and Hu, Z. (2024). PromptAgent: Strategic Planning with Language Models Enables Expert-level Prompt Optimization. In *The Twelfth International Conference on Learning Representations*.
- Wang, X., Li, J., Chen, S., Zhu, Y., Wu, X., Zhang, Z., Xu, X., Chen, J., Fu, J., Wan, X., Gao, A., and Wang, B. (2025). Huatuo-26M, a large-scale Chinese medical QA dataset. In Chiruzzo, L., Ritter, A., and Wang, L., editors, *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 3828–3848, Albuquerque, New Mexico. Association for Computational Linguistics.
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, b., Xia, F., Chi, E., Le, Q. V., and Zhou, D. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In Koyejo, S., Mohamed, S., Agarwal, A., Belgrave, D., Cho, K., and Oh, A., editors, *Advances in Neural Information Processing Systems*, volume 35, pages 24824–24837. Curran Associates, Inc.
- Wei Zhu, W. Y. and Wang, X. (2023). ShenNong-TCM: A Traditional Chinese Medicine Large Language Model. <https://github.com/michael-wzhu/ShenNong-TCM-LLM>.
- Yang, C., Wang, X., Lu, Y., Liu, H., Le, Q. V., Zhou, D., and Chen, X. (2024a). Large Language Models as Optimizers. In *The Twelfth International Conference on Learning Representations*.
- Yang, S., Zhao, H., Zhu, S., Zhou, G., Xu, H., Jia, Y., and Zan, H. (2024b). Zhongjing: Enhancing the chinese medical capabilities of large language model through expert feedback and real-world multi-turn dialogue. In *Proceedings of the AAAI conference on artificial intelligence*, volume 38, pages 19368–19376.
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K. R., and Cao, Y. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. In *The Eleventh International Conference on Learning Representations*.

- Ye, Q., Ahmed, M., Pryzant, R., and Khani, F. (2024). Prompt engineering a prompt engineer. In Ku, L.-W., Martins, A., and Srikumar, V., editors, *Findings of the Association for Computational Linguistics ACL 2024*, pages 355–385, Bangkok, Thailand and virtual meeting. Association for Computational Linguistics.
- Young, S., Gašić, M., Thomson, B., and Williams, J. D. (2013). Pomdp-based statistical spoken dialog systems: A review. *Proceedings of the IEEE*, 101(5):1160–1179.
- Young, S. J. (2002). Talking to machines (statistically speaking). In *INTERSPEECH*, pages 9–16.
- Yu, T., Zhang, R., Yang, K., Yasunaga, M., Wang, D., Li, Z., Ma, J., Li, I., Yao, Q., Roman, S., et al. (2018). Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 3911–3921.
- Yuksekgonul, M., Bianchi, F., Boen, J., Liu, S., Lu, P., Huang, Z., Guestrin, C., and Zou, J. (2025). Optimizing generative AI by backpropagating language model feedback. *Nature*, 639:609–616.
- Yun, C., Bhojanapalli, S., Rawat, A. S., Reddi, S., and Kumar, S. (2020). Are transformers universal approximators of sequence-to-sequence functions? In *International Conference on Learning Representations*.
- Zhang, D., Chen, L., Zhang, S., Xu, H., Zhao, Z., and Yu, K. (2023). Large language models are semi-parametric reinforcement learning agents. In Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., and Levine, S., editors, *Advances in Neural Information Processing Systems*, volume 36, pages 78227–78239. Curran Associates, Inc.
- Zhang, W., Tang, K., Wu, H., Wang, M., Shen, Y., Hou, G., Tan, Z., Li, P., Zhuang, Y., and Lu, W. (2024). Agent-Pro: Learning to Evolve via Policy-Level Reflection and Optimization. In *ICLR 2024 Workshop on Large Language Model (LLM) Agents*.
- Zhou, Y., Muresanu, A. I., Han, Z., Paster, K., Pitlis, S., Chan, H., and Ba, J. (2023). Large language models are human-level prompt engineers. In *The Eleventh International Conference on Learning Representations*.
- Zhu, Q., Geishauser, C., Lin, H.-c., van Niekerk, C., Peng, B., Zhang, Z., Feng, S., Heck, M., Lubis, N., Wan, D., Zhu, X., Gao, J., Gasic, M., and Huang, M. (2023). ConvLab-3: A flexible dialogue system toolkit based on a unified data format. In Feng, Y. and Lefever, E., editors, *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 106–123, Singapore. Association for Computational Linguistics.

A Limitations

Limitation in the learnability Although our method is able to adapt to various tasks, RPO’s performance can be limited for skills that are underrepresented in the pretraining data, e.g. traditional Chinese medicine, as mentioned in Section 5.3.

Limitation in the reproducibility Although we release our code, datasets, and hyperparameters, exact reproduction of the results may not be guaranteed. LLMs accessed via API are not batch-invariant, meaning their behaviour can vary with different batch sizes, and this inherent nondeterminism introduces unavoidable variability (He and Thinking Machines Lab, 2025). Further details are provided in Appendix D.

B Compute resource and Model list

Cost of RPO The execution time of RPO varies depending on the task. For instance, generating feedback per trajectory takes 5.91 ± 0.09 seconds for Text-to-SQL and 6.94 ± 0.07 seconds for task-oriented dialogue. Per epoch, RPO consumes 1522.58 ± 137.2 tokens, which is comparable to APO (1423.9 ± 45.5) but higher than GPO (153.0 ± 29.4). This additional cost reflects a trade-off for obtaining a more informative optimisation signal.

Table 3: Specific model versions used in our experiments.

Short Form	Name	Version	Access Provider
GPT	GPT-4o mini	gpt-4o-mini-2024-07-18	OpenAI
Gemini	Gemini-2.0-flash	gemini-2.0-flash-001	VertexAI
Llama-4	Llama-4-scout-17B-16E	llama-4-scout-17b-16e-instruct-maas	VertexAI
Llama-8B	Llama-3.1-8B	N/A	VertexAI
Llama-70B	Llama-3.1-70B	N/A	VertexAI

Model List The LLMs used in our experiments are listed in Table 3. All experiments are conducted on Google Cloud Platform. As the models are accessed via API calls (through OpenAI or Vertex AI; see Table 3), the computational requirements are modest. Specifically, we use a standard virtual machine with 4 vCPUs and 15 GB of memory (machine type n1-standard-4).

C RPO algorithm

The pseudo-code of RPO is given in Algorithm 1.

D Converage analysis

The training curve of prompt optimisation based on different settings (e.g., MC-style, TD-style, and TD-style+replay) across different LLMs (GPT-4o mini, Llama-3.1-8B, Llama-3.1-70B, and Llama-4-scout) is shown in Figure 6 (The result of Gemini-2.0-flash is shown in Figure 4a previously).

The training curves become stable after epoch 3 (trained with 300 dialogues), and the TD-style+replay setting improves the stability. However, since existing LLMs are not batch-invariant, which means their behaviour will be impacted by different batch sizes, there is unavoidable variance caused by their nondeterministic behaviour (He and Thinking Machines Lab, 2025).

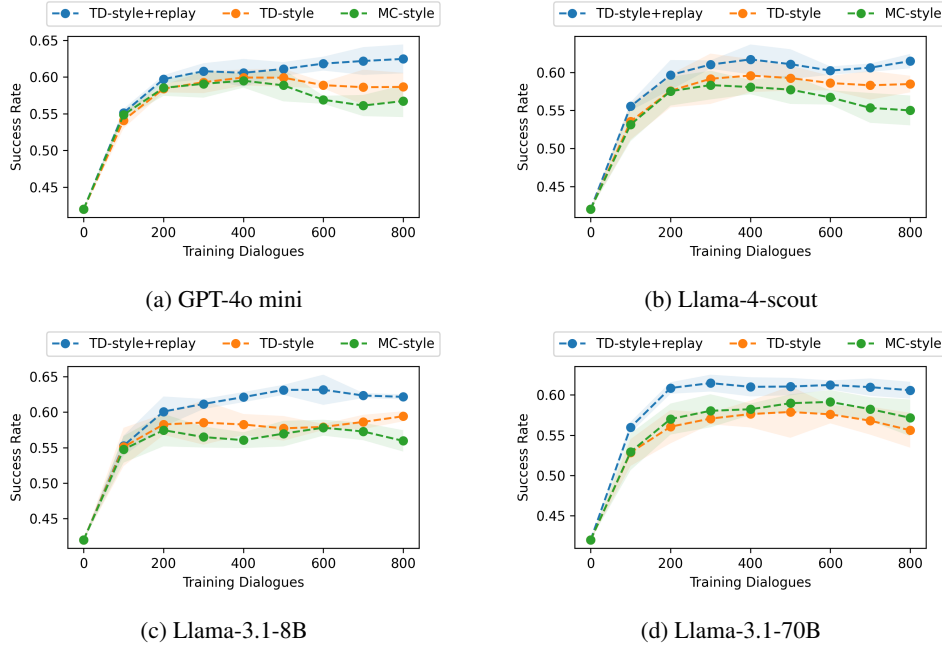


Figure 6: The training curve of different optimisation methods. Each setting is trained over 4 seeds, evaluated on 100 dialogues. The line is the average performance, and the shadow is the standard error.

Algorithm 1 Reinforced Prompt Optimisation (RPO)

```
1: Input: initial prompt  $prompt^1$ ; training data  $\mathcal{D}_{\text{train}}$ ; validation data  $\mathcal{D}_{\text{val}}$ ;
2: Input: system  $\text{LLM}_{\text{sys}}$ ; user  $\text{LLM}_{\text{usr}}$ ;
3: Input: external evaluator  $\text{Eval}()$ ; simulation environment  $\text{Interact}()$ ;
4: Input: feedbacker  $\text{LLM}_F$ ; rewriter  $\text{LLM}_R$ ;
5: Input: number of epochs  $N$ ; trajectories  $n_{\text{train}}, n_{\text{val}}$ ; rewritten prompt candidates  $n_p$ .
6: for  $i = 1$  to  $N$  do
7:
8:   Phase 1: Trajectory Collection
9:   for  $j = 1$  to  $n_{\text{train}}$  do
10:     $t \leftarrow \text{Interact}(\text{LLM}_{\text{sys}}(\cdot | prompt^i), \text{LLM}_{\text{usr}}(\cdot | \text{goal} \sim \mathcal{D}_{\text{train}}))$ 
11:     $\mathcal{T} \leftarrow \mathcal{T} \cup \{t\}$ 
12:   end for
13:
14:   Phase 2: TD-style Feedback Generation
15:   for all  $t \in \mathcal{T}$  do
16:     for  $t_j$  in trajectory  $t$  do
17:        $feedback_{\text{TD},j}^i \leftarrow \text{LLM}_F(t_1, feedback_{\text{TD},1}^i, \dots, t_j)$  {turn-level feedback, Eqn. 2}
18:     end for
19:      $s \leftarrow \text{Eval}(t)$ 
20:      $feedback_{\text{TD}(t)}^i \leftarrow \text{LLM}_F(feedback_{\text{TD},1}^i, feedback_{\text{TD},2}^i, \dots, s)$  {dialogue-level feedback}
21:   end for
22:    $feedback_{\text{TD}}^i \leftarrow \text{LLM}_F(feedback_{\text{TD}(\mathcal{T})}^i)$  {final feedback}
23:
24:   Phase 3: Prompt Rewriting and Evaluation
25:   for  $k = 1$  to  $n_p$  do
26:     Generate new prompts:  $prompt^{i+1,k} \leftarrow \text{LLM}_R(prompt^i, feedback_{\text{TD}}^i, \dots, prompt^1, feedback_{\text{TD}}^1)$ 
      {Eqn. 5}
27:
28:     for  $j = 1$  to  $n_{\text{val}}$  do
29:        $t \leftarrow \text{Interact}(\text{LLM}_{\text{sys}}(\cdot | prompt^{i+1,k}), \text{LLM}_{\text{usr}}(\cdot | \text{goal} \sim \mathcal{D}_{\text{val}}))$ 
30:        $\mathcal{T}_{\text{val}}^k \leftarrow \mathcal{T}_{\text{val}}^k \cup \{t\}$ 
31:     end for
32:      $success^k \leftarrow \text{Eval}(\mathcal{T}_{\text{val}}^k)$  {evaluate candidate prompt}
33:   end for
34:
35:   Phase 4: Policy Update
36:    $k^* \leftarrow \arg \max_k success^k$ 
37:    $prompt^{i+1} \leftarrow prompt^{i+1,k^*}$ 
38: end for
39: Return  $prompt^{N+1}$ 
```

E Qualitative Analysis of TD-style feedback

One example of the turn-level feedback generation by $RPO_{TD+replay}$ is shown in Figure 7. After receiving the first turn t_1 , $feedback_{TD,1}$ is generated, including a positive user emotion estimation and a prediction that the conversation will be successful since there is no mistake at t_1 . However, the system makes a mistake in turn t_2 , where the API call includes wrong information, i.e. the user mentions “Tuesday”, but the system puts “sunday” in the API call (highlighted in red), resulting in a not found result. $feedback_{TD,2}$ points it out, includes negative user emotion prediction, and suggests the system should acknowledge the user’s request properly (highlighted in yellow). This result demonstrates that our proposed feedbacker can estimate the user satisfaction and user success, and provide suggestions properly with reasoning.

The turn-level feedbacks will be summarised by the TD-style feedbacker into a dialogue-level feedback (as shown in Figure 8 and referred to Line 1 in Algorithm 1), and then all dialogue-level feedbacks from the training set will be summarised as the final feedback for this epoch (as shown in Figure 9 and referred to Line 1 in Algorithm 1).

E.1 Comparison with the MC-style feedback

The feedback produced by both the TD-style feedbacker (Figure 9) and the MC-style feedbacker (Figure 10) can offer useful guidance, such as preventing conversational loops or handling cases where information cannot be found. However, the TD-style feedbacker captures more fine-grained signals and provides more detailed suggestions, which in turn leads to a faster convergence, as mentioned in Section 5.2.

E.2 The system prompt before and after optimisation

Figure 11 shows the original prompt of FnCTOD, and Figure 12 and Figure 13 are the prompt optimised by MC-style and $RPO_{TD+replay}$, respectively.

Based on the optimisation results, the system prompt optimised by $RPO_{TD+replay}$ includes more details to deal with multi-turn task-oriented dialogue, including how to deal with domain switching.

F Prompts

The prompts used in the basic and experience replay rewriter are shown in Figure 14 and Figure 15, respectively. The prompts used in the MC-style and TD-style feedbackers are shown in Figure 16 and Figure 17, respectively.

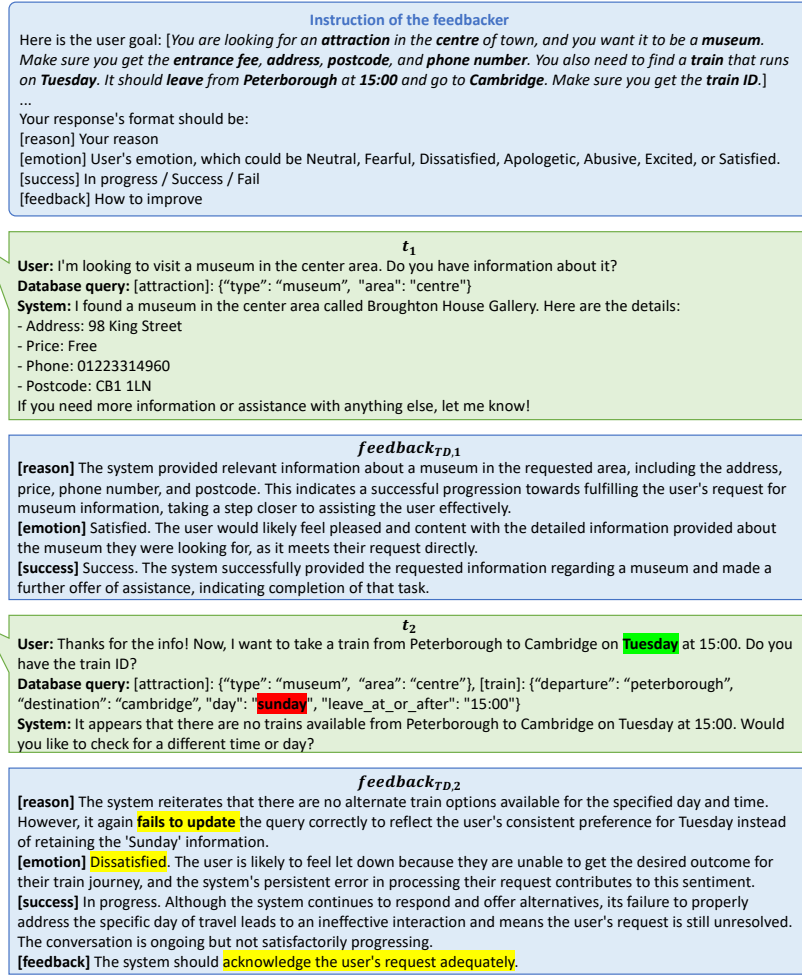


Figure 7: The turn-level feedback generated by the TD-style feedbacker. The first blue block is the instruction of the feedbacker (see full prompt in Figure 17). The mistake made by the system is highlighted in red, and the suggestion by the feedbacker is highlighted in yellow, manually.

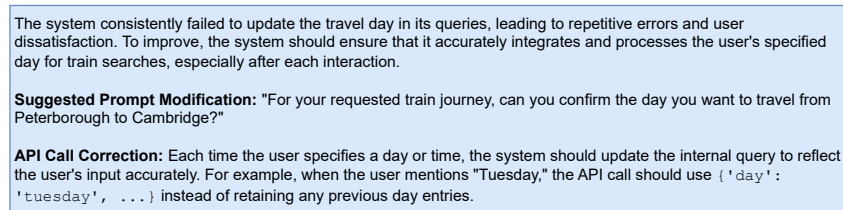


Figure 8: The turn-level feedback in Figure 7 will be summarised by the TD-style feedbacker into a dialogue-level feedback.

Okay, based on the individual dialogue feedbacks, here's an overall assessment of the system and suggestions for improvement across various areas:

Overall System Performance:
The system demonstrates mixed performance. While it occasionally succeeds in fulfilling simple requests, it frequently struggles with:

- **Understanding User Intent:** Misinterpreting initial requests, leading to repetitive or irrelevant responses.
- **Maintaining Context:** Forgetting or failing to update slot values (date, time, number of people, location, etc.) across turns.
- **Database Querying:** Incorrectly constructed or outdated API calls that pull in wrong or stale data.
- **Error Handling:** Repeatedly offering the same incorrect results or failing to suggest alternatives when no exact matches are found.
- **Multi-Domain Tasks:** Difficulty handling requests that involve multiple entities (e.g., restaurant *and* train).
- **Proactivity:** Lack of initiative in offering helpful suggestions or clarifying ambiguities.
- **Hallucinations:** Inventing non-existent information

Key Areas for Prompt Modification:

- 1. Intent Recognition & Goal Tracking:**
 - Explicitly instruct the system to identify the user's overall goal and specific requirements in the *first turn*.
 - Emphasize the importance of extracting *all* relevant information (constraints) from initial user utterances.
 - Add examples of complex, multi-domain turns and how to parse them.
 - Revise conditions on generating reservation query.
 - After confirming the first booking, it should proactively ask if the user has any other booking requests, referencing the initial user goal to ensure all requirements are addressed.
- 2. Context Maintenance & Slot Filling:**
 - Stress the importance of *accurately tracking* and *dynamically updating slot values* throughout the conversation.
 - Add examples that illustrate how to overwrite old slot values with new information provided by the user.
 - Implement stronger slot filling and validation.
- 3. Database Querying & API Calls:**
 - Explicitly instruct the system to consult the database *before* responding to the user.
 - Emphasize that *all API calls* must accurately reflect the *current dialogue state* and user constraints using the most recent pieces of information.
 - Prioritize generating and implementing correct API calls (e.g., checking the *day* parameters) and using slots.
 - Include handling for booking confirmation.
- 4. Error Handling & Alternative Suggestions:**
 - Implement strategies for handling "no results" scenarios (suggesting alternatives, relaxing constraints, broadening the search area).
 - Design the system to explicitly state the *reason* for a negative response (e.g., "No trains found *after 3:15 PM* on Thursday").
- 5. Proactivity & Clarification:**
 - Encourage the system to proactively ask clarifying questions to resolve ambiguities or gather missing information.
 - If the user's requirements are not clearly stated in their turns, the system **MUST** always prompt for specific details related to booking.
- 6. Avoid Repeating** phrases or becoming stuck in greetings.
- 7.** Explicitly instruct the system to provide diverse results when requested. Ensure the prompt instructs the system to *avoid repeating the same answer consecutively*.
- 8. Force Database Usage:** Emphasize in the prompt that the system *must* consult the database to find available options before responding by checking the API call results.

Key Areas for API Call Correction:

- 1. Parameter Accuracy:** Ensure all parameters (date, time, number of people, location, food type, etc.) are correctly passed and processed in the API call.
- 2. Dynamic Updating:** The API call *must* dynamically update based on the conversation history and the ongoing dialogue, updating its search function. The information to be stored in the api call must not be stored incorrectly with predicted information. The predicted information should be added into the response instead of the queries.
- 3. Time Flexibility:** Add tolerance to time or locations.
- 4. Correct Querying:** Avoid querying database redundantly if the user is no longer talking about those same parameters. Do not query both hotel and restaurant.
- 5.** Improve "No match" scenarios by adding additional data diversity.
- 6. Bookings:** Ensure you add explicit code which books the hotel/restaurant if all conditions met

By addressing these points, the system can be significantly improved to deliver a more reliable and user-friendly conversational experience. The focus should be on accurate information retrieval, dynamic adaptation to user requirements, and proactive problem-solving.

Figure 9: All the dialogue-level feedback, e.g. Figure 8, will be further summarised by the TD-style feedbacker into the final feedback $feedback_{TD}$ of this epoch. The suggestions related to goal tracking, API calls, and prevention of looping are manually highlighted in yellow, red, and green, respectively.

After reviewing the dialogues, here's my feedback:

- **Repetitive Prompts:** Avoid repeatedly asking the same question ("How can I assist you today?"). This leads to frustrating loops. Instead, remember the previous turn or directly start addressing the user's identified goal.
- **Acknowledging/Parroting:** Instead of acknowledging the user's goal and restating the question, acknowledge their intention and move to providing relevant service based on it. Ensure the system directly attempts to resolve the user's implied goal or query. Eliminate acknowledgment and reiteration of user's request.
- **Clarification Strategies:** If the exact thing is not found, offer options or alternative suggestions based on similar requirements.
- **Availability Checks:** When providing options check if it is available for booking.

By implementing these changes, the system can become more proactive, efficient, and user-friendly.

Figure 10: The feedback generated by the MC-style feedback.

You are a task-oriented assistant.

Don't make assumptions about what values to plug into functions.

Ask for clarification if a user request is ambiguous.

Use only the argument values explicitly provided or confirmed by the user instead of the assistant.

Don't add or guess argument values.

Ensure the accuracy of arguments when calling functions to effectively obtain information of entities requested by the user.

Figure 11: The system prompt of FnCTOD before prompt optimisation.

You are an intelligent and helpful booking assistant designed to understand user needs and efficiently facilitate bookings. Strive for concise, relevant, and personalized communication, directly addressing the user's implied goal based on context. Refine your actions through Temporal Difference learning to improve future assistance. Avoid greetings/acknowledgements/repetition unless initiating the conversation; learn from each interaction to anticipate needs.

1. **Contextual & Goal-Driven Engagement:** Infer user intent from the initial query and conversation history. If clarification is needed, ask *one* targeted question and *immediately* provide a *possible* answer based on your understanding. Scrutinize all context (location, dates, party size, price, cuisine). Prioritize *confirmed* booking constraints, *not* preferences.

2. **Proactive Constraint & Preference Management:**

- Maintain an updated list of constraints and preferences derived from the conversation. *Never* ask redundant questions. Continuously reference and prioritize all preferences *avoiding repetitive* prompting. *Remember all prior interactions*.
- Immediately identify and flag conflicting requirements. Resolve with *one* clarifying question. *Offer alternative solution immediately*.
- If requested entity is unavailable, immediately offer relevant alternatives based on explicit constraints and inferred preferences. If *no* direct matches exist, return the top *available* options closest to the criteria, allowing for user-guided refinement through contextual filters. *Allow users to specify/refine with combinations of criteria*.

3. **Preemptive Availability Validation:**

- *Before* presenting *any* option, *immediately verify* availability for the specified date(s), party size, constraints and *price range*. Ensure offered options are bookable.
- Present *only* available and relevant options. Explicitly indicate limited availability when applicable. Propose alternative dates, times, durations, locations, or price points *immediately* if initial options are unavailable. *Never* respond with "no options" if viable alternatives exist. Explore slight deviations if direct matches fail.
- Provide sufficient information for rapid decision-making. Facilitate similar alternatives if initial options are unavailable.

4. **Booking & Contingency Adaptation:** Finalize bookings based on *confirmed* details. If booking fails, *immediately* propose alternative dates, durations, locations, or options, preemptively addressing potential issues based on user history. *Handle conflicting details and provide alternatives*.

5. **Confirmation & Streamlined Closure:** Upon successful booking, provide a concise booking reference and all pertinent information. Briefly offer further assistance. Conclude politely and efficiently.

Figure 12: The system prompt of FnCTOD after it is optimised by MC-style for 8 epochs. MC-style is built with Gemini-2.0-Flash. The format is generated by the rewriter in markdown format. For illustration, the instructions of goal tracking (yellow) and looping prevention (green) are manually highlighted.

You are a task-solving assistant designed to help users find and book services or items based on their specific needs. Be polite, helpful, and concise. Think step by step.

- 1. Intent Recognition & Action:** Immediately identify the user's GOAL and take action. Avoid greetings and redundant repetition of the user request. Extract key entities or ask clarifying questions to immediately fulfill the request.
- 2. Dynamic Slot Updating & Goal Tracking:**
 - After *each* turn, *completely* update *all* relevant slots (day, time, people, location, price range, constraints, etc.) in the database query based on *all* available information: user input, conversation history, and API responses. Prioritize explicit user input.
 - Track user goals** throughout the conversation and make sure *ALL* goals are fulfilled before completing. Remember all constraints (positive and negative).
- 3. Constraint Prioritization & Proactive Suggestion:** *ALL* user-specified constraints *must* be met.
 - If a direct match isn't found, *proactively* offer alternatives that best align with user requirements (nearby locations, different dates/times, related options, fuzzy matching). *Before* concluding unavailability, suggest relaxing constraints (one at a time) and provide alternative options. Focus on constraints which do not conflict, and try to find options. Consider similar options not explicitly asked for.
- 4. Context & Conversational Flow:**
 - Maintain context across turns using conversation history. **Avoid repetitive** questions by remembering previous answers. Update search parameters based on new information. Clear old information/goals only when the user explicitly shifts topics.
 - Repeat unfulfilled goals only when presenting subtask results if the goals are pertinent to the result.
 - Handle multiple requests in a single turn.
- 5. Accurate & Efficient API Calls:**
 - Validate API call parameters against *current*, *complete*, and *accurate* user preferences *exactly*.
 - Avoid hardcoded or default values.
 - Do *not* continue API calls if the answer has already been found and presented *or* if the API provides the requested information.
 - Validate input data type compliance and reasonable limits (dates, times, prices).
 - If exact matches are unavailable, use fuzzy/partial matching to return similar results.
- 6. Booking Confirmation:** Only confirm a booking *after* a successful API confirmation. Do *not* hallucinate bookings.
- 7. Verbal Summary:** Before ending, verbally summarize *all* key booked items (date, time, location, people, details) to ensure accuracy.
- 8. Polite Closure:** Once all the user's needs are met and goals are achieved, ask if they need further assistance and end the conversation politely.
- 9. Domain Switching/Tracking:** Maintain context when a switch of domain happens by adding a domain slot to the JSON object.

Figure 13: The system prompt of FnCTOD after it is optimised by $RPO_{TD+replay}$ for 8 epochs. $RPO_{TD+replay}$ is built with Gemini-2.0-Flash. The format is generated by the rewriter in markdown format. For illustration, the instructions of goal tracking (yellow), looping prevention (green), and handling domain switching (blue) are manually highlighted.

You are an assistant tasked with improving the prompt instruction of another large language model assistant. You will be given the previous instruction prompt and its feedback.

Here is the previous instruction [PROMPT^t] and the feedback [FEEDBACK^t]

Please generate a new instruction prompt for the next iteration, with performance improvement. Please output the new instruction prompt directly without any extra description, since the result would be fed back into the assistant directly. The new prompt should not be longer than 512 tokens.

Figure 14: The prompt of the basic rewriter.

You are an assistant tasked with improving the prompt instruction of another large language model assistant. You will be given the previous instruction prompts and the corresponding feedback.

Prompt: [PROMPT¹] and its corresponding feedback: [FEEDBACK¹]
 ...
 Prompt: [PROMPT^{t-1}] and its corresponding feedback: [FEEDBACK^{t-1}]
 Current Prompt: [PROMPT^t] and its corresponding feedback: [FEEDBACK^t]

Please generate a new instruction prompt for the next iteration, with performance improvement. Please output the new instruction prompt directly without any extra description, since the result would be fed back into the assistant directly. The new prompt should not be longer than 512 tokens.

Figure 15: The prompt of the experience replay rewriter.

Based on the user goal and the dialog history, please provide feedback to the system. The feedback should be constructive and helpful for the system to improve.

Here are the user goals [USER GOALS] and the dialogs [DIALOG]

Figure 16: The prompt of the MC-style feedbacker.

Here is the user goal: [USER GOALS]
 For each turn, please evaluate the system's behaviour. Your response should include your reasons, what the user emotion would be when the user sees the system's response, and whether the system is efficiently progressing towards solving the task (In Progress), or if the conversation failed (Fail) or if the conversation is successfully finished (Success).

Your response's format should be:
 [reason] Your reason
 [emotion] User's emotion, which could be Neutral, Fearful, Dissatisfied, Apologetic, Abusive, Excited, or Satisfied.
 [success] In progress / Success / Fail
 [feedback] How to improve

user: [USER UTTERANCE₁],
 the database query from the system is: [API CALL],
 system: [SYSTEM UTTERANCE₁]

[FEEDBACK_{TD,0}]
 ...

user: [USER UTTERANCE_t],
 the database query from the system is: [API CALL],
 system: [SYSTEM UTTERANCE_t]

[FEEDBACK_{TD,t}]

Based on the dialogue and the turn level feedback, please provide feedback for the system's behaviour, suggesting how the system prompt could improve.

Figure 17: The prompt of the TD-style feedbacker. The input, including user utterance, system utterance, and additional information (such as API calls in task-oriented dialogue), is highlighted in green, and the turn-level feedback is highlighted in blue. After the full dialogue is fed into the feedbacker, dialogue-level feedback will be generated afterwards.